

# Repository & Execution

Vom Prozessbild zum steuerbaren System –  
Single Source of Truth, Governance und ausführbare Workflows.

*Lehrskript – Tag 4 von 16*

Lernpfad:  
Wissen → Anwendung → Management

Dozent:  
Can Siebert

Bearbeitungsdauer:  
1 Kurstag

Format:  
Theorie · Fallstudie · Coaching

---

## Lernziele dieses Tages

An den ersten drei Tagen haben Sie Prozesse aus mehreren Perspektiven *modelliert* – als Notation, als Wertstrom, als Datenrealität, mit Blick auf Stakeholder, KPIs und SOA. Heute machen wir den entscheidenden Schritt von der einzelnen Diagrammdatei zum *betriebenen System*: einem Repository, das Modelle steuerbar macht, und einer Execution-Schicht, die Prozesse ausführbar und auditierbar werden lässt.

### L1 – Wissen (Reproduktion und Einordnung)

- Den Repository-Ansatz mit seinen Bestandteilen (Prozesslandkarte, Modelle, Rollen, Dokumente, Versionen, KPIs, Risiken) einordnen.
- Modell-Governance benennen: Owner, Modeler, Reviewer, Approver – und das Zusammenspiel mit Benennungs-, Versions- und Freigaberegeln.
- Die Differenz zwischen *Business-BPMN* (verständlich, kommuniziert Verantwortung) und *Execution-BPMN* (präzise, maschinen-nah) erklären.
- Camunda-typische Konstrukte einordnen: User Task, Service Task, Timer-/Message-/Error-Event, Datenobjekte und Variablen.

### L2 – Anwendung (Transfer auf konkrete Situationen)

- Eine Repository-Struktur in drei Ebenen (E2E-Prozess → Subprozess → Arbeitsdokument) für einen Beispielprozess entwerfen.
- Einen Mini-Prozess in einen *Automations-Slice* schneiden: Start, Ende, Datenminimum, Ausnahmepolitik definieren.
- Variantenführung bewusst entscheiden: separate Modelle oder Variantenattribute – mit Trade-off-Begründung.
- KPIs für ein Workflow-Programm definieren (Durchlaufzeit, Eskalationsquote, Rework) und Owner zuordnen.

### L3 – Management (Entscheidung und Verantwortung)

- Repository als *Betriebssystem* für Prozesssteuerung verstehen – nicht als Ablage.
- Execution als *Vertrag* zwischen Fach, IT und Compliance behandeln: Daten, Fehler, SLAs, Eskalation.
- Entscheidungen unter Unsicherheit treffen: MVP statt Perfektion, mit Frühwarnsignalen abgesichert.
- Governance-Regeln auf das Minimum reduzieren, das 80 % des Chaos verhindert – und alles weitere bewusst delegieren.

#### Roter Faden

Modelle leben oder sterben *nach* der Modellierung – in einem Repository mit Governance, oder ausgeführt in einer Engine mit Audit Trail. Wer modelliert, ohne den Lebenszyklus mitzudenken, baut Diagramme für Schubladen. Senior-Praxis heißt: *Repository steuert Transparenz, Execution steuert Durchlaufzeit – und beide brauchen denselben Owner.*

## 1. Einstieg: Vom Diagramm zum steuerbaren System

Es gibt eine bemerkenswerte Konstante in fast jeder Organisation, die seit ein paar Jahren modelliert hat: Es existieren *viele* Diagramme. Sie liegen in SharePoint-Ordern, in Visio-Dateien, in Powerpoint-Anhängen, in PDF-Exporten. Was meist fehlt, ist die Fähigkeit, sie als *System* zu betreiben. Niemand weiß, welches die aktuelle Version ist; niemand weiß, wer entscheidet, ob eine Änderung freigegeben wird; niemand weiß, ob das, was im Diagramm steht, mit dem übereinstimmt, was die Workflow-Engine ausführt. (Dumas et al., 2018, Kap. 1; Harmon, 2019)

Der heutige Tag adressiert genau diese Lücke. Modellieren ist eine notwendige Bedingung, aber keine hinreichende, um Prozesse zu *steuern*. Hinreichend wird es erst durch zwei Schichten, die typischerweise unterschätzt werden: eine **Repository-Schicht**, die Modelle als verwaltete Artefakte hält, und eine **Execution-Schicht**, die ausgewählte Modelle tatsächlich ausführbar macht.

### 1.1 Drei Symptome eines fehlenden Repository

1. **Niemand findet die aktuelle Version.** Ein Auditor fragt, welcher Stand verbindlich ist. Drei Antworten von drei Personen, alle leicht unterschiedlich.
2. **Standorte modellieren divergent.** Jede Niederlassung benennt anders, modelliert in unterschiedlichen Tiefen, dokumentiert auf eigenen Laufwerken. Vergleichbarkeit ist Glücksspiel.
3. **Änderungen passieren "irgendwie".** Eine Anpassung des Genehmigungsworkflows landet als E-Mail-Notiz im Postfach, nicht als versionierter Change Request mit Reviewer-Verantwortung.

Diese Symptome sind nicht primär Tool-Probleme. Sie sind *Governance-Probleme*, die durch ein gutes Tool unterstützt – aber nicht ersetzt – werden können.

### 1.2 Was ein Repository leisten muss

Ein **Prozess-Repository** ist mehr als ein Ordner mit Diagrammen. Es ist eine kuratierte Sammlung von Artefakten mit definierten Metadaten, Verantwortlichkeiten und Lebenszyklen. Im Kern braucht es: (Scheer, 2002; Rosemann & vom Brocke, 2010)

- Eine **Prozesslandkarte** (Value Chain), die die End-to-End-Sicht ordnet – typischerweise 8 bis 15 Kernprozesse auf der obersten Ebene.
- Die zugehörigen **Prozessmodelle** (BPMN, EPC, Activity), Subprozesse und Arbeitsdokumente.
- **Rollen:** Process Owner, Modeler, Reviewer, Approver. Pflicht, nicht optional.
- **Metadaten:** Owner, Gültigkeit, Status (Draft, Review, Approved, Retired), Version, letzte Änderung, Standort, Risikoklasse.
- Optional: Verlinkung zu **KPIs, Risiken, Controls** – damit das Repository nicht nur Diagramme hält, sondern *Steuerungsobjekte*.

#### Eselsbrücke: P-M-R-V

Ein Repository steht auf vier Beinen, das fünfte ist der Owner:

**Prozesslandkarte** – die Übersicht über alle Kernprozesse.

**Modelle** – BPMN, EPC, Subprozesse, Arbeitsdokumente.

**Rollen** – Owner, Modeler, Reviewer, Approver.

Versionen – Status, Gültigkeit, Änderungslog.

*Wer ein Bein vergisst, steht auf einem Tisch, der wackelt. Wer alle fünf hat, hat ein Betriebssystem für Prozesse.*

### 1.3 Was Execution leisten muss

Wenn das Repository der “Bibliothekar” ist, ist die Execution-Schicht die “Maschine”. Sie nimmt ausgewählte Modelle und führt sie aus – als Workflows mit User Tasks, Service Tasks, Timern, Eskalationen und Audit Trail. (Freund & Rücker, 2019; OMG, 2014)

Nicht jedes Modell muss ausführbar sein. Im Gegenteil: Die wertvollsten Modelle sind oft die *Übersichtsmodelle*, die Verantwortung sichtbar machen und Kommunikation strukturieren. Aber bestimmte Teilstrecken – typischerweise solche mit klarer Start-Ende-Definition, eindeutigen Daten und nachvollziehbaren Fehlerfällen – profitieren massiv davon, in einer **Workflow-Engine** (Camunda, Flowable, jBPM, oder vergleichbare BPMS-Produkte) zu laufen.

## 2. Repository-Design: Single Source of Truth in drei Ebenen

Ein praxistaugliches Repository kommt mit einer überraschend einfachen Struktur aus. Drei Ebenen reichen für den Start – mehr Ebenen erzeugen Pflegeaufwand, der den Nutzen oft übersteigt.

### 2.1 Die drei kanonischen Ebenen

1. **Ebene 1 – E2E-Prozess.** Die Sicht auf einen vollständigen Wertschöpfungsstrang: “Order-to-Cash”, “Incident-to-Resolution”, “Lead-to-Order”, “Hire-to-Retire”. Auf dieser Ebene wird *nicht* im Detail modelliert, sondern Scope, Owner, KPI und übergeordneter Verlauf geklärt.
2. **Ebene 2 – Subprozess.** Die Detail-Modelle, die zeigen, wie ein Teil des E2E-Prozesses tatsächlich abläuft. Hier wohnt das eigentliche BPMN/EPC-Diagramm mit Lanes, Gateways und Tasks. Pro E2E-Prozess typischerweise 3 bis 8 Subprozesse.
3. **Ebene 3 – Arbeits- und Nachweisdokument.** Die operativen Artefakte: SOP (Standard Operating Procedure), Checklisten, Formulare, Anweisungen, Schulungsunterlagen, Auditnachweise. Sie hängen an einzelnen Subprozessen oder Tasks.

### 2.2 Die Mindestmenge an Metadaten

Wer Modelle vergleichbar halten will, muss Pflichtfelder definieren. *Fünf* Felder sind ein guter Anfang – mehr ist Komfort, weniger ist Schluderei: (Allweyer, 2020; Harmon, 2019)

- **Owner.** Genau eine Person (Rolle), nicht ein Verteiler. “Wer eskaliert, ruft Owner an.”
- **Gültigkeit.** Ab wann gilt diese Version, bis wann ist sie aktuell, wann ist Review fällig?
- **Status.** Draft, Review, Approved, Retired. Keine “halben” Zustände dazwischen.
- **Standort/Variante.** Welche Organisation, welche Region, welche Produkt-Variante?
- **Risikoklasse.** Hoch (compliance- oder geldkritisch), Mittel, Niedrig – steuert Review-Tiefe.

### Owner ist eine Person, nicht eine Abteilung

Ein Process Owner ist eine *konkrete Rolle* mit Namen, nicht “Bereich Vertrieb”. Wer als Owner einen Verteiler einträgt, hat de facto keinen Owner. Im Audit-Fall stellt sich heraus: niemand fühlt sich zuständig, weil alle gemeint waren und keiner verantwortlich hat.

## 2.3 Variantenführung – als Attribut oder als eigenes Modell?

Die schwierigste Repository-Entscheidung ist meist nicht die Struktur, sondern die *Variantenpolitik*. Drei Standorte machen das Onboarding leicht unterschiedlich; zwei Produktlinien folgen ähnlichen, aber nicht identischen Prozessen. Sollen das separate Modelle sein, oder ein Modell mit Variantenattributen?

Faustregel:

- **Variantenattribut** – wenn 80 % des Modells identisch ist und nur Detail-Felder (Schwellwert, Approver-Rolle, SLA) variieren. Vorteil: Pflege einmal, Konsistenz hoch. Nachteil: Modell wirkt überladen.
- **Separates Modell** – wenn Hauptpfade abweichen, andere Akteure beteiligt sind oder regulatorische Unterschiede bestehen. Vorteil: Klarheit. Nachteil: Pflegeaufwand multipliziert sich.

## 2.4 Repository-Blueprint – ein Beispiel

Für einen E2E-Prozess “Order-to-Cash” kann ein realistisches Repository so aussehen:

---

|                |                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Ebene 1</b> | Order-to-Cash (Owner: Head of Sales Ops, KPI: DLZ 7 Tage, Status: Approved v3.1)                                |
| <b>Ebene 2</b> | Auftragsannahme, Verfügbarkeitsprüfung, Kreditcheck, Auslieferung, Fakturierung, Mahnwesen                      |
| <b>Ebene 3</b> | SOP Verfügbarkeitsprüfung, Eskalationsleitfaden Kreditcheck, Mahnstufen-Brieftexte, Audit-Nachweis Fakturierung |

---

Diese Sicht ist nicht spektakulär – aber sie ist *vollständig*. Sie macht sichtbar, wer im Notfall anzurufen ist, welche Version gilt, und wo das nächste Detail liegt. Das ist der Unterschied zwischen einer Ordnerstruktur und einem Steuerungsinstrument.

## 3. Governance: Rollen, Reviews und Freigaben

Ohne Rollen wird ein Repository binnen Monaten zur Abstellkammer. *Mit* Rollen, klaren Regeln und einem ehrlichen Review wird es zum Steuerungssystem. Die Verteilung darf dabei pragmatisch sein: vier Rollen sind in der Praxis ausreichend, mehr macht es schwerfällig. (Rosemann & vom Brocke, 2010)

### 3.1 Die vier Kernrollen

- **Process Owner.** Verantwortet den E2E-Prozess fachlich, vertritt ihn nach außen, entscheidet über strategische Änderungen. Hängt an Ebene 1.
- **Modeler.** Modelliert konkret, in Abstimmung mit den Fachbereichen. Beherrscht das Handwerk. Hängt an Ebene 2.
- **Reviewer.** Prüft Modelle gegen Konventionen, GoM und 7PMG. Mindestens vier Augen pro Approval.

- **Approver.** Gibt finale Modellversionen frei. Häufig der Process Owner selbst, oder ein Bereichsleiter mit fachlicher Verantwortung.

### 3.2 Der Freigabeprozess in drei Stufen

Ein einfacher und in jeder Organisation tragfähiger Lebenszyklus:

1. **Draft.** Modeler arbeitet am Modell, schreibt Annahmen sichtbar in Annotations.
2. **Review.** Reviewer prüft auf Konsistenz, Vollständigkeit, Konventionen. Feedback zurück an Modeler, ggf. mehrere Iterationen.
3. **Approved.** Approver gibt frei. Version wird mit Datum, Reviewer-Namen und Approver-Namen versiegelt. Nur diese Version ist verbindlich für Operation und Audit.

Hinzu kommt ein vierter Status: **Retired**, wenn ein Modell durch eine neuere Version oder einen veränderten Prozess obsolet wird. Wichtig: *Retired-Modelle nicht löschen*, sondern archivieren – für Audit-Rückfragen.

#### Eselsbrücke: D-R-A-R

Vier Stationen im Modell-Lebenszyklus, in dieser Reihenfolge:

**Draft** – Modeler arbeitet, Annahmen sichtbar.

**Review** – Reviewer prüft gegen Konvention.

**Approved** – Approver versiegelt mit Datum und Namen.

**Retired** – archivieren, nicht löschen.

*Wer eine Station überspringt, sammelt Schulden im Repository – die spätestens im Audit fällig werden.*

### 3.3 Modellierungs-Policy auf einer Seite

Eine effektive **Modellierungs-Policy** passt auf eine Seite und regelt das Minimum. Mehr ist Bürokratie:

1. Notation: BPMN 2.0 für ausführungsnahe Modelle, EPC oder Activity für Übersichten.
2. Benennung: Verb + Objekt für Aktivitäten/Funktionen, Zustand für Ereignisse, “X-to-Y” für E2E-Prozesse.
3. Pflicht-Metadaten: Owner, Status, Version, Gültigkeit, Risikoklasse.
4. Pflicht-Elemente in jedem Modell: Start- und End-Ereignis, mindestens eine Lane mit klarer Rolle, jede Gateway-Bedingung beschriftet.
5. Review-Pflicht: vor jedem Approved-Status, ohne Ausnahme.
6. Variantenregel: Attribute bevorzugt, separate Modelle nur bei begründeter Hauptpfad-Divergenz.
7. Update-Rhythmus: jedes Modell mindestens jährlich reviewen, bei Prozess-Änderungen sofort.

### 3.4 Die zwei Regeln, die 80 % des Chaos verhindern

Wenn man nur zwei Governance-Regeln durchsetzen darf, sind es diese: (Harmon, 2019)

- *Jedes Modell hat einen genannten Owner und einen Approved-Status mit Datum.* Das eliminiert Versions-Verwirrung und Verantwortungs-Vakuum.
- *Änderungen laufen über einen Change Request mit Reviewer.* Das eliminiert die “mal eben angepasst”-Kategorie, die Audit-Findings produziert.

Alles weitere ist Komfort und kann später ergänzt werden, wenn die Organisation gelernt hat, mit dem Minimum zu leben.

## 4. Business-BPMN vs. Execution-BPMN

Eine der häufigsten Quellen für Frust in Modellierungs-Projekten ist die unsichtbare Annahme, dass *ein* Modell gleichzeitig dem Vorstand etwas erklären und der Workflow-Engine etwas vorgeben kann. In der Praxis sind das zwei verschiedene Modelle, die denselben Prozess beschreiben – mit unterschiedlicher Tiefe und Strenge. (Freund & Rücker, 2019; OMG, 2014)

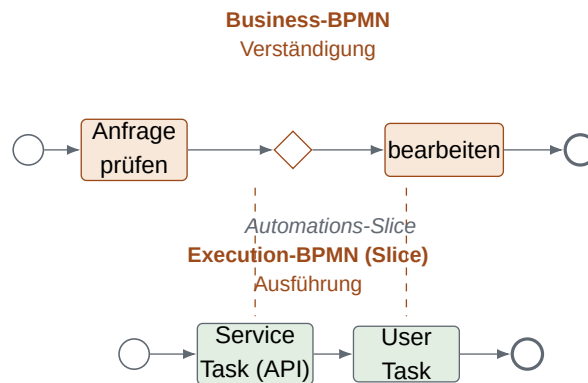


Abbildung 1: Vom Business-BPMN zum ausführbaren Slice.

### 4.1 Business-BPMN: verständlich, kommuniziert Verantwortung

Ein **Business-BPMN**-Modell ist für die *Fachseite* optimiert. Es darf an manchen Stellen unscharf sein, solange es:

- den Hauptpfad sauber zeigt,
- Verantwortlichkeiten über Lanes sichtbar macht,
- Entscheidungspunkte (Gateways) eindeutig benennt,
- Schnittstellen zu anderen Pools markiert,
- Annahmen explizit dokumentiert.

Ziel ist Verständigung – nicht Vollständigkeit. Ein Business-BPMN kann an einzelnen Stellen Sub-Prozesse nutzen, ohne sie zu detaillieren, oder Manual Tasks beschreiben, ohne sie ausführbar zu machen.

### 4.2 Execution-BPMN: präzise, maschinen-nah

Ein **Execution-BPMN**-Modell muss in eine Workflow-Engine laden und dort *ausgeführt* werden können. Das verlangt eine andere Disziplin: jede Bedingung formal, jeder Pfad vollständig, jede Variable typisiert. Konkret kommen folgende Elemente in den Vordergrund:

- **User Task** – ein menschlicher Schritt mit Form, Zuweisungsregel und SLA.
- **Service Task** – ein Systemschritt, typischerweise ein API-Aufruf oder eine Datenbankoperation.
- **Send/Receive Task** – expliziter Nachrichtenaustausch über System- oder Pool-Grenzen.
- **Business Rule Task** – delegiert Entscheidung an eine Decision Engine (DMN).
- **Timer Event** – Reminder, Eskalation, Deadlines (“nach 3 Tagen erinnern, nach 5 Tagen

eskalieren”).

- **Message Event** – asynchrone Auslöser, z. B. Eingang eines externen Events.
- **Error Event** – Fehlerbehandlung mit Boundary Events an User/Service Tasks.
- **Datenobjekte und Prozess-Variablen** – typisierte Datenstruktur, die zwischen Tasks weitergegeben wird.

### 4.3 Gegenüberstellung

|                      | Business-BPMN                          | Execution-BPMN                      |
|----------------------|----------------------------------------|-------------------------------------|
| <b>Adressat</b>      | Fachseite, Management, Audit-Übersicht | Workflow-Engine, IT, Operation      |
| <b>Tiefe</b>         | Hauptpfad, wichtige Varianten          | jeder Pfad vollständig durchführbar |
| <b>Gateways</b>      | Benennung der Entscheidung             | formale Bedingung in FEEL/Skript    |
| <b>Tasks</b>         | generisch (“Rechnung prüfen”)          | typisiert (User/Service/Script/BR)  |
| <b>Daten</b>         | implizit oder annotiert                | Variablen mit Typ, Datenobjekte     |
| <b>Fehlerfälle</b>   | “geht zurück an Sachbearbeitung”       | Error Boundary Event, Eskalation    |
| <b>Versionierung</b> | semantisch (v3.1 = Geschäftsänderung)  | technisch (Deployment-Version)      |

Beide Modelle haben Existenzrecht. Es ist Senior-Praxis, sie *nicht* zu vermischen: das eine ist ein Kommunikationsobjekt, das andere ein Ausführungsobjekt. Die Brücke ist eine **Übergabe-Routine**, die definiert, welche Teile des Business-BPMN in Execution-BPMN überführt werden – und welche bewusst nicht.

#### Zwei Modelle, ein Owner

Business- und Execution-BPMN sollten denselben *Owner* haben. Wenn “die Fachseite” das Business-Modell pflegt und “die IT” das Execution-Modell, driften beide auseinander, sobald Änderungen kommen. Geteilte Verantwortung ist Geheimnis um Verantwortung.

## 5. Den automatisierbaren Slice finden

Nicht jeder Prozess gehört in eine Engine. Die Kunst besteht darin, in einem dokumentierten Prozess die Teilstrecke zu identifizieren, die *realistisch* und *wertvoll* automatisierbar ist. Diese Teilstrecke nennen wir den **Automations-Slice**. (vom Brocke & Mendling, 2021)

### 5.1 Sechs Kriterien für einen guten Slice

1. **Klarer Start und klares Ende.** Was triggert den Slice, wann ist er fertig? Wenn diese Grenzen verschwimmen, wird der Workflow zur Sammlung von Sonderlocken.
2. **Eindeutige Daten.** Welche Felder kommen rein, welche werden zwischen Tasks weitergegeben, welche werden geschrieben? Ohne Datenklarheit kann keine Service Task funktionieren.
3. **Bekannte Fehlerfälle.** Was passiert bei Timeout, fehlenden Daten, abgelehntem Antrag? Mindestens die häufigsten drei Fehlerpfade müssen vor Go-Live definiert sein.
4. **Hohe Frequenz und niedrige Varianz.** Ein Workflow, der hundertmal pro Tag in fast immer derselben Form läuft, lohnt sich. Ein Workflow für drei Sonderfälle pro Jahr ist eine Checkliste.
5. **Vorhandene Schnittstellen.** Wenn die Service Tasks auf nicht existierende APIs zugreifen



sollen, ist die Automatisierung ein API-Projekt mit Workflow-Dekoration – das muss man wissen.

6. **Messbarer Nutzen.** Eine KPI vor Go-Live (Durchlaufzeit, Fehlerquote, Personenstunden) und ein realistischer Zielwert.

## 5.2 Beispiel: Rechnungsfreigabe als Slice

Nehmen wir den klassischen Mini-Prozess “Rechnungsfreigabe”:

1. Rechnung kommt per E-Mail.
2. Buchhaltung erfasst Grunddaten.
3. Wenn Betrag > 5.000 €, muss Teamleitung freigeben.
4. Wenn Kostenstelle fehlt, geht es zurück an die Anforderungsstelle.
5. Nach Freigabe wird im ERP gebucht und Zahlung angestoßen.
6. Bei Fristüberschreitung wird erinnert und nach 3 Tagen eskaliert.

## 5.3 Slice-Analyse

Wir markieren typisiert: *User Tasks* (Menschenarbeit), *Service Tasks* (Systemaktionen), *Entscheidungen*, *Fehlerfälle*, *Fristen*.

- Schritt 1 (Rechnung per E-Mail) – *Trigger* (Message Event aus Mailbox), evtl. Vorverarbeitung *Service Task*.
- Schritt 2 (Grunddaten erfassen) – *User Task* Buchhaltung, mit Pflichtfeldern (Betrag, Kostenstelle, Lieferant).
- Schritt 3 (Schwellwert-Entscheidung) – *Gateway* (Business Rule Task möglich).
- Schritt 3a (Freigabe Teamleitung) – *User Task* mit SLA.
- Schritt 4 (Kostenstelle fehlt) – *Error Path* mit Rückgabe an Anforderungsstelle.
- Schritt 5 (ERP-Buchung) – *Service Task* (API ERP).
- Schritt 5b (Zahlung) – *Service Task* (API Payment).
- Schritt 6 (Frist) – *Timer Boundary Event* an User Task, mit Reminder nach 24 h und Eskalation nach 72 h.

Datenminimum (drei Felder, die der Workflow zwingend braucht): **Betrag, Kostenstelle, Freigeber-Rolle.**

## 5.4 Ausnahmepolitik – bewusst nicht automatisieren

Es ist Senior-Praxis, im ersten Release nicht alle Ausnahmen abzubilden. Eine sinnvolle Ausnahmepolitik benennt explizit, welche Fälle *organisatorisch* und nicht *technisch* abgefangen werden. Beispiele für die Rechnungsfreigabe:

- Lieferanten ohne Stammdatensatz: erst manuell in ERP anlegen, dann Workflow neu starten.
- Fremdwährungen: vorläufig nicht automatisiert, manueller Prozess.
- Sammelrechnungen mit mehreren Kostenstellen: vorläufig manuell, in Release 2 einplanen.

**Eselsbrücke: S-D-F-F**

Vor jedem Automations-Slice prüfen:

Start/Ende – klar abgrenzbar?

Daten – Minimum-Set definiert?

Fehlerfälle – mindestens drei dokumentiert?

Frequenz – hoch genug, dass es sich lohnt?

*Wenn auch nur ein S-D-F-F-Buchstabe wackelt, ist es kein Slice, sondern ein Wunschzettel.*

## 6. KPIs für Repository und Workflows

Was nicht gemessen wird, wird nicht gesteuert. Aber: was falsch gemessen wird, erzeugt Fehlalarme. KPIs für Repository und Execution sollten zugleich *wenig*, *wirksam* und *nicht gameable* sein. (Reinkemeyer, 2020)

### 6.1 KPI-Kandidaten für das Repository

- **Approval-Quote.** Anteil der Modelle in Status “Approved” (Ziel: > 80 %). Niedrig → Reviews stocken.
- **Review-Aktualität.** Anteil der Modelle mit Review jünger als 12 Monate. Niedrig → Repository altert.
- **Owner-Vollständigkeit.** Anteil der Modelle mit eindeutig benanntem Owner (Ziel: 100 %).

### 6.2 KPI-Kandidaten für Workflow-Execution

- **Durchlaufzeit (DLZ).** Zeit von Start- bis End-Event. Owner: Process Owner. Achtung: SLA-Definition muss vorher klar sein.
- **Eskalationsquote.** Anteil der Cases, die mindestens einmal eskaliert wurden. Steigend → Slice zu eng geschnitten oder Ressourcen fehlen.
- **Rework-Quote.** Anteil der Cases mit Korrekturschleife. Steigend → Datenqualität oder Pflichtfelder am Eingang prüfen.
- **Auto-Resolution-Rate.** Anteil vollständig automatisch abgeschlossener Cases ohne manuelle Intervention.

### 6.3 Anti-Gaming und Owner

Jede KPI braucht einen Owner – und ein Bewusstsein dafür, wie sie verfälscht werden könnte:

| KPI                     | Mögliches Gaming                              | Gegenmaßnahme                |
|-------------------------|-----------------------------------------------|------------------------------|
| <b>DLZ</b>              | Cases früh “vorläufig” schließen              | gekoppelt mit Rework-Quote   |
| <b>Eskalationsquote</b> | Eskalationen umgehen oder umetikettieren      | Audit-Trail-Check Stichprobe |
| <b>Auto-Resolution</b>  | einfache Cases bevorzugen, schwere blockieren | Verteilung nach Komplexität  |
| <b>Approval-Quote</b>   | “Quick Approvals” ohne echte Reviews          | Reviewer-Vier-Augen-Pflicht  |

**KPI braucht Kontext**

Eine KPI ohne Owner ist Statistik. Eine KPI ohne Gegenkontrolle ist Einladung zum Gaming. Eine KPI ohne Maßnahmen-Verknüpfung ist Dekoration. Senior-Steuerung verlangt: Owner, Gegenkontrolle, Maßnahmen-Verknüpfung – für jede einzelne Kennzahl.

## 7. Fallstudie: MediSupply Services – E2E-Repository und Pilot

Wir betrachten ein realistisches Szenario, das die Themen des Tages verbindet – Repository-Design, Pilot-Auswahl, Automations-Slice und KPI-Steuerung. *MediSupply Services* ist ein B2B-Anbieter im Medizinprodukte-Umfeld mit mehreren Standorten. Die Probleme sind typisch: uneinheitliche Prozesse, schwankende Durchlaufzeiten, lückenhafte Audit-Nachweise. Ein Teilprozess soll schnell automatisiert werden, aber IT-Kapazität ist knapp.

### 7.1 Ausgangslage und Restriktionen

- Zeit: 8 Wochen für Pilot.
- Budget: 90 000 €.
- ERP-Schnittstelle: existiert, aber nicht vollständig dokumentiert.
- Standorte: drei in DACH, jeweils mit eigenen Varianten.

### 7.2 Schritt 1 – E2E-Prozesslandkarte

Wir entwerfen eine Landkarte mit fünf Kernprozessen:

1. *Lead-to-Order* – Anfrage bis Auftragsbestätigung.
2. *Order-to-Delivery* – Auftragsabwicklung bis Auslieferung.
3. *Invoice-to-Cash* – Rechnungsstellung bis Zahlungseingang.
4. *Quality-to-CAPA* – Qualitätsereignis bis Korrekturmaßnahme.
5. *Hire-to-Onboarding* – Personalprozess.

### 7.3 Schritt 2 – Pilot wählen

Wir wählen **Lead-to-Order** als Pilot. Begründung:

- Hohe Frequenz (mehrere hundert Cases pro Monat).
- Klare Start-Ende-Definition (Anfrage rein, Bestätigung raus).
- Existierende Stammdaten (Kunde, Produkt) – Datenminimum greifbar.
- Sichtbare DLZ-Probleme (Beschwerden aus Vertrieb).

Verworfen Alternative: *Invoice-to-Cash*. Zwar hohe Wirkung auf Cashflow, aber abhängig von der unklaren ERP-Schnittstelle – zu hohes Risiko im 8-Wochen-Zeitraum.

### 7.4 Schritt 3 – Repository-Struktur für den Pilot

- Ebene 1: “Lead-to-Order” (Owner: Head of Sales Ops, KPI-Set: DLZ, Konversionsrate, Reklamationsquote).
- Ebene 2 (Subprozesse): Anfrageannahme, Bonitäts-/Verfügbarkeitscheck, Angebotserstellung, Vertragsabschluss, Auftragsanlage.
- Ebene 3: SOP Anfrageannahme (Pflichtfelder), Eskalationsleitfaden Bonität, Angebots-Templates

pro Produktkategorie.

Metadaten-Standard (fünf Pflichtfelder): Owner, Gültigkeit, Status, Standort, Risikoklasse. Zusätzlich: KPI-Verlinkung und letzte Änderung mit Approver-Name.

## 7.5 Schritt 4 – Automationskandidat identifizieren

Innerhalb von Lead-to-Order ist der Subprozess *Bonitäts- und Verfügbarkeitscheck* der heißeste Kandidat:

- Start: Anfrage liegt vor, Kundendaten erfasst.
- Service Tasks: Bonitätsanfrage an externen Provider (sync), Verfügbarkeitscheck an ERP (sync).
- XOR: Bonität ok? Verfügbar?
- User Task bei Sonderfall: Sales-Mitarbeiter validiert manuell.
- Ende: Ergebnis-Datensatz für Angebotserstellung übergeben.

## 7.6 Schritt 5 – Execution-Skizze (Camunda-Style)

**Start** (Message Event “Anfrage geprüft”)

→ *Service Task “Bonität abfragen”* (API extern, Timeout 5 s, 2 Retries)

→ *Service Task “Verfügbarkeit prüfen”* (API ERP, Idempotent, Case-ID)

→ **XOR-Gateway** “Beide ok?”

*Ja:* → *Service Task “Ergebnis speichern”* → **Ende** “Check positiv”

*Nein:* → *User Task “Manuelle Prüfung”* (SLA 4 h, Eskalation an Sales-Lead)

→ *XOR “Freigegeben?”* → **Ende** “Check positiv” oder “Check negativ”.

Fehlerfälle: API-Timeout (Retry, dann manuelle Prüfung), unbekannter Kunde (zurück an Sales mit Hinweis), ERP nicht verfügbar (Timer Event, Wiederholung nach 15 min, dann Eskalation).

## 7.7 Schritt 6 – KPIs für den Pilot

- **DLZ Bonitäts-/Verfügbarkeitscheck** – Ziel: < 30 min in 90 % der Fälle. Owner: Process Owner Lead-to-Order.
- **Eskalationsquote manuelle Prüfung** – Ziel: < 15 %. Owner: Sales-Lead.
- **Rework-Quote** – Cases, die nach Check zurück an Sales gehen. Ziel: < 5 %. Owner: Sales Ops.

## 7.8 Lessons aus der Fallstudie

- Pilotwahl ist kein Lostopf, sondern eine Bewertung nach Frequenz, Klarheit, Datenverfügbarkeit, Schnittstellen-Reife.
- Variantenpolitik wird früh, nicht spät, entschieden – sonst wachsen drei Modelle parallel.
- Automation ohne KPI ist Selbstzweck. KPI ohne Owner ist Statistik. Owner ohne Maßnahmenrecht ist Theater.

# 8. Management-Perspektive: Repository als Betriebssystem, Execution als Vertrag

Auf Wissens-Ebene sind Repository und Execution Notations- und Werkzeug-Themen. Auf Anwendungs-Ebene sind sie Handwerk. Auf Management-Ebene sind sie *Operating-Model-Entscheidungen* – mit Auswirkungen auf Verantwortlichkeiten, Kosten, Risiken und Kultur.

## 8.1 Zwei zentrale Zielkonflikte

1. **Governance-Streng vs. Akzeptanz.** Strenge Reviews und Freigabeprozesse erzeugen Qualität – und Widerstand. Zu lockere Regeln erzeugen Wildwuchs. Praxis: Minimum-Set definieren, alles darüber bewusst entscheiden – nicht “alles oder nichts”.
2. **Standardisierung vs. Standortautonomie.** Wenn jeder Standort seine Variante behalten will, gibt es keine Vergleichbarkeit. Wenn ein zentrales Modell alle Standorte zwingt, geht lokale Klugheit verloren. Praxis: *Guardrails statt Wildwuchs* – harte Regeln für 5 nicht verhandelbare Standards, lokale Freiheit innerhalb dieser Leitplanken.

## 8.2 Entscheidungen unter Unsicherheit

In Repository- und Workflow-Initiativen sind die Entscheidungen mit langer Halbwertszeit besonders teuer:

- **Tool-Stack.** Einmal etabliert, kostet ein Tool-Wechsel mindestens ein Jahr. Vorher klären: braucht es ein BPMS, oder reicht eine Modellierungs-Plattform mit Workflow-Anschluss?
- **Owner-Modell.** Wer ist End-to-End-Owner – ein Fachbereich, ein Querschnittsamt, oder eine Linien-Rolle? Diese Entscheidung prägt die nächsten drei bis fünf Jahre.
- **Variantenpolitik.** “Ein Modell für alle” vs. “ein Modell pro Standort” – jede Antwort hat sechsstellige Folgekosten in Pflege.

Für jede dieser Entscheidungen gilt: schriftlich treffen, mit verworfener Alternative dokumentieren, mit Frühwarnsignal versehen (“wenn Bedingung X eintritt, prüfen wir die Entscheidung erneut”).

## 8.3 Risikoarchitektur eines Workflow-Programms

1. **Daten-Risiko.** Workflow scheitert an Datenqualität. Mitigation: Pflichtfelder im Eingangs-Formular, Service-Task “Datenvalidierung” vor jedem Service Task mit externem System.
2. **Schnittstellen-Risiko.** APIs ändern sich, ohne dass es jemand merkt. Mitigation: Versionierung, Contract Tests (konzeptionell), API-Owner mit Change-Notification-Pflicht.
3. **Compliance-Risiko.** Audit-Findings durch fehlende Trails. Mitigation: jedes Approved-Modell mit Datum/Approver, jeder Workflow-Lauf mit unveränderlichem Audit-Log.
4. **Akzeptanz-Risiko.** Operative Teams umgehen den Workflow durch Schatten-Prozesse. Mitigation: frühzeitige Einbindung, “Was wird für mich besser?”-Argumentation, klare Eskalationswege.

## 8.4 Senior-Shift: vom Tool-Verstehen zur Verantwortungsarchitektur

Der Wechsel von “ich modelliere ein Diagramm” zu “ich verantworte ein Steuerungssystem” erkennt man an drei Kennzeichen:

- Man entscheidet die Variantenpolitik vor dem ersten Tool-Klick.
- Man kennt die zwei Governance-Regeln, die 80 % des Chaos verhindern – und verteidigt sie auch unter Druck.
- Man kann in 90 Sekunden gegenüber einem Vorstand begründen, warum genau dieser Slice

automatisiert wurde und nicht ein anderer.

#### Schluss-Merksatz für Tag 4

Ein Repository ohne Owner ist eine Ablage. Eine Execution ohne Audit Trail ist eine Black Box. Ein Workflow ohne KPI ist eine Verlagerung des Problems. Senior-Praxis verbindet alle drei: *Owner steuert das Repository, Audit Trail sichert die Execution, KPI macht beides messbar.*

## 9. Take-aways aus Tag 4

1. **Repository = Betriebssystem.** Drei Ebenen (E2E → Subprozess → Arbeitsdokument), fünf Pflicht-Metadaten, vier Rollen – mehr braucht es zum Start nicht.
2. **Execution = Vertrag.** Daten, Fehler, SLAs, Eskalation – jeder Punkt schriftlich, sonst trägt im Ernstfall niemand.
3. **Business-BPMN ≠ Execution-BPMN.** Zwei Modelle, ein Owner, eine definierte Übergabe-Routine.
4. **Slice statt All-in.** Ein klar abgegrenzter Automations-Slice mit messbarem Nutzen schlägt jede “großen Wurf”-Ankündigung.
5. **Zwei Governance-Regeln verhindern 80 % Chaos.** Owner + Approved-Status mit Datum sowie Change Requests mit Reviewer.
6. **Variantenpolitik früh entscheiden.** Attribute oder separate Modelle – die Wahl prägt Pflegekosten über Jahre.
7. **KPI braucht Owner + Anti-Gaming.** Jede Kennzahl mit verantwortlicher Person und Gegenkontrolle, sonst entsteht Reporting ohne Steuerung.
8. **Guardrails statt Wildwuchs.** Zentrale Standards für das Minimum, lokale Freiheit innerhalb – Standortautonomie ohne Beliebigkeit.

## 10. Literatur

---

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

**Allweyer, T. (2020).**

*BPMN 2.0 – Business Process Model and Notation: Einführung in den Standard für die Geschäftsprozessmodellierung* (4. Aufl.). BoD.

**Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018).**

*Fundamentals of business process management* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>

**Freund, J., & Rücker, B. (2019).**

*Real-life BPMN: Includes an introduction to DMN* (4th ed.). CreateSpace.

**Harmon, P. (2019).**

*Business process change: A business process management guide for managers and process professionals* (4th ed.). Morgan Kaufmann.

**Object Management Group. (2014).**

*Business Process Model and Notation (BPMN), Version 2.0.2* (formal/2013-12-09). <https://www.omg.org/spec/BPMN/2.0.2/>

**Reinkemeyer, L. (Hrsg.). (2020).**

*Process mining in action: Principles, use cases and outlook*. Springer. <https://doi.org/10.1007/978-3-030-40172-6>

**Rosemann, M., & vom Brocke, J. (2010).**

The six core elements of business process management. In J. vom Brocke & M. Rosemann (Hrsg.), *Handbook on business process management 1* (S. 107–122). Springer. [https://doi.org/10.1007/978-3-642-00416-2\\_5](https://doi.org/10.1007/978-3-642-00416-2_5)

**Scheer, A.-W. (2002).**

*ARIS – Vom Geschäftsprozess zum Anwendungssystem* (4. Aufl.). Springer.

**vom Brocke, J., & Mendling, J. (Hrsg.). (2021).**

*Business process management cases: Digital innovation and business transformation in practice* (Vol. 2). Springer. <https://doi.org/10.1007/978-3-662-63047-1>

## 11. Glossar

---

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

**Approval-Quote**

Anteil der Modelle in Status “Approved”. Indikator für Governance-Aktivität.

**Approver**

Rolle, die finale Modellversionen freigibt. Häufig der Process Owner.

**Audit Trail**

Unveränderliches Log aller Schritte und Entscheidungen eines Workflow-Laufs. Pflicht in regulierten Umfeldern.

**Automations-Slice**

Klar abgegrenzte Teilstrecke eines Prozesses, die als Workflow ausgeführt wird. Erfordert Start/Ende, Datenminimum, Fehlerfälle.

**BPMS**

*Business Process Management System*. Kombiniert Modellierung, Ausführung, Überwachung und Optimierung in einer Plattform.

**Business-BPMN**

BPMN-Variante für Fach- und Management-Sicht. Hauptpfade, Lanes, Annotations – nicht zwingend ausführbar.

**Camunda**

Verbreitete Workflow-Engine, die BPMN 2.0 ausführt. Open-Core mit Enterprise-Erweiterungen.

**Change Request**

Strukturierter Antrag zur Änderung eines Modells, mit Reviewer und Freigabe.

**Datenminimum**

Kleinste Menge an Feldern, die ein Workflow zwingend braucht, um ausgeführt zu werden.

**Definition of Done (DoD)**

Liste prüfbarer Kriterien, ab denen ein Modell oder Workflow als fertig gilt.

**E2E-Prozess**

End-to-End-Prozess: vollständiger Wertschöpfungsstrang von Trigger bis Outcome.

**Eskalationsquote**

Anteil der Cases, die mindestens einmal eskaliert wurden. KPI für Workflow-Steuerung.

**Execution-BPMN**

BPMN-Variante für Workflow-Engine. Jeder Pfad vollständig, jede Bedingung formal, Tasks typisiert.

**Freigabeprozess**

Lifecycle eines Modells: Draft → Review → Approved → Retired.

**Governance**

Regeln, Rollen und Prozesse, die Modelle steuerbar und auditierbar machen.

**Guardrails**

Verbindliche Leitplanken, innerhalb derer lokale Autonomie zulässig ist.

**MVP**

*Minimum Viable Product*. Erste Version, die Lerneffekt erzeugt – nicht alle Features.



**Owner (Process Owner)**

Person mit fachlicher End-to-End-Verantwortung für einen Prozess. Pflicht-Metadatum jedes Modells.

**Repository**

Verwaltete Sammlung von Prozessmodellen und Artefakten mit Metadaten, Versionen und Lebenszyklus.

**Service Task**

BPMN-Element: ein Schritt, der durch ein System ausgeführt wird, typischerweise API-Aufruf.

**SLA**

*Service Level Agreement*. Vereinbarung über Antwort- oder Durchlaufzeiten.

**Status**

Zustand eines Modells im Lebenszyklus: Draft, Review, Approved, Retired.

**Timer Event**

BPMN-Element für zeitgesteuerte Aktionen: Reminder, Eskalation, Deadlines.

**User Task**

BPMN-Element: ein Schritt, der durch einen Menschen ausgeführt wird, typischerweise mit Form.

**Variantenattribut**

Mechanismus, um Modellvarianten als Datenfeld am gemeinsamen Modell zu führen.